MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

DEPARTMENT OF HUMANITIES AND MANAGEMENT
MANIPAL – 576104, KARNATAKA, INDIA

Capital Markets Technology Lab Individual Report

BACHELOR OF TECHNOLOGY

Submitted by

Aarav Ashutosh Joshi 230958164

Under the guidance of

Dr. Varun Sarda
Associate Professor,
Department of Humanities and Management,
MIT Manipal

Dr. Prakash Kalingrao Aithal
Assistant Professor - Senior Scale,
School of Computer Engineering,
MIT Manipal

Dr. Shilpa Shetty H
Assistant Professor,
Department of Humanities and Management,
MIT Manipal

Dr. Harsh Nagar
Assistant Professor,
School of Computer Engineering,
MIT Manipal

Github Repository Link-Arrowsincoming24/capitalmarketslab_with_testsuite

Academic Year 2025-26

1. Executive Summary

This report presents a detailed technical account of Group 1's work in designing, implementing, and stabilising the core order lifecycle path of a FIX-based exchange backend. The module spans the complete processing chain, beginning with FIX and REST protocol intake, continuing through order validation, orchestration, and matching, and ending with durable persistence, operational query support, and stress-tested reliability. The final system is not only functionally complete but also structured to preserve lifecycle correctness under normal trading flow as well as under high-churn cancellation and modification scenarios.
As Group 1 leader, I was responsible not only for my individual module assignment, but also for coordinating the overall architecture, integration direction, and validation process across the group. My leadership role involved ensuring that each member's work aligned to a single coherent order lifecycle design, that interfaces between modules remained consistent, and that implementation decisions supported both correctness and testability. I helped drive the team toward a layered architecture in which intake, orchestration, matching, persistence, and API control were cleanly separated while still working together as a unified order management pipeline. This coordination was essential because the system's behaviour depended on tight interaction between several components, and a weakness in any one layer could break the end-to-end flow.

The individual contribution documented in this report corresponds to module assignment G1-M4, which carried primary responsibility for Order Persistence and Database Integration. This was a central architectural role because all correctness guarantees exposed to clients through FIX execution reports, REST responses, and external scenario assertions ultimately depend on the persistence layer maintaining an accurate and query able representation of order state. My work in this area covered the entity model, schema design, index strategy, DAO and repository operations, batch persistence methods, and lifecycle-safe update handling. The goal was to ensure that state was not only stored but stored in a way that reflected the true business lifecycle of each order, including status transitions, filled quantity, leaves quantity, and average price.

A major part of this contribution was aligning database structure with operational access patterns. I designed the order schema so that it could support both transactional correctness and practical query performance, especially for views by symbol, status, client, and time. I also implemented repository-level persistence methods that allow the system to create, update, and retrieve order records reliably, while batch-oriented methods improve throughput and reduce persistence overhead when dealing with bursts of activity. This was important because the trading simulator does not only process isolated requests; it must remain stable and efficient during rapid sequences of order submissions, cancellations, and modifications.
In addition to the persistence role, I made a significant secondary contribution to scenario stabilisation, particularly in the stress scenario family T30 through T35, with special attention to T33, the cancel storm scenario. This work went beyond simple bug fixing. It required understanding how asynchronous trading events interact with persistence timing and scenario assertions, then tightening the system so that race-prone behaviour no longer produced unstable or misleading results. I worked to align persistence write timing with the expectations of the scenario framework and used a disciplined regression approach, repeatedly running the full 35-scenario suite until the results were consistently stable. This ensured that the system was not just passing on a single run but was genuinely reliable under repeated validation.

2. Problem Context and Engineering Goal

2.1 The Problem with Lifecycle Inconsistency in Trading Systems

Trading systems are difficult to get right because the most serious failures are often not obvious crashes, but silent inconsistencies in order state. A duplicate order can create an unintended position, a cancel request can appear successful while the order remains active, and a partial fill can be visible in memory without being written correctly to storage. These problems are especially dangerous because they may not show up in simple manual testing. Instead, they often appear only when the full order lifecycle is exercised repeatedly under realistic operating conditions.

The main problem addressed by this project is lifecycle inconsistency across intake, matching, cancel, and persistence layers. Group 1 was responsible for designing an order service in which lifecycle state is treated as durable, canonical, and authoritative from the moment an order is accepted. Every important transition is recorded and updated in a controlled way so that the database, the order book, and the scenario harness all reflect the same truth. This approach makes the system easier to audit, easier to test, and much more reliable under stress.

This issue becomes even more important in a trading environment because the system must stay consistent even when orders are being processed quickly. The external scenario suite was useful in exposing these kinds of problems, especially in cancel-heavy and stress-oriented workflows. That is why the design was not limited to making the system functionally correct once, but to making it remain correct across repeated scenario runs and race-prone conditions.

2.2 Technical Goals of the Project

The project was built around a small set of hard technical goals, and each one was treated as a required part of the final solution.

- First, the system had to validate and deduplicate intake requests at the protocol boundary. A duplicate client order ID must never create two active order records, because that would break identity correctness and lead to conflicting business state.
- Second, accepted orders had to be persisted immediately, rather than only after matching was complete. This ensures that every valid order has a durable record as soon as it is accepted, even if execution happens later or only partially.
- Third, fill and status fields had to remain mathematically consistent after matching. The system needed to preserve the relationship between filled quantity, remaining quantity, and original quantity so that stored state always matched actual execution progress.
- Fourth, cancel, amend, and bulk operations had to behave as guarded state transitions rather than open-ended updates. Terminal-state orders should not be mutated as if they were still active, and invalid transitions had to be rejected cleanly.
- Finally, the system had to remain stable across the full external scenario suite, including the stress cases. This meant the implementation had to survive not just happy-path order submission, but also high-volume and race-prone activity such as cancel storms and mixed operation bursts.

3. Group 1 Scope and Team-Level Architecture

3.1 Scope Boundary

Group 1 was assigned ownership of the complete order handling pipeline, meaning the team was responsible for everything that happens after an order enters the system and before its final lifecycle state is permanently recorded. In practical terms, this included the flow from protocol or API ingress, through validation, orchestration, matching, persistence, and control operations, all the way to terminal outcomes such as FILLED, CANCELED, or REJECTED.

The boundary was intentionally end-to-end rather than component-specific, because in a trading system the real correctness requirement is not whether each individual module works in isolation, but whether the entire lifecycle behaves consistently from beginning to end. This scope was important because it made Group 1 accountable for lifecycle correctness, not just code ownership. Once an order is accepted, the system must ensure that every major transition is reflected in the durable record, so that the state visible to FIX execution flows, REST queries, and external scenario checks is always aligned. That means the team was not only building entry points or utility functions but also ensuring that accepted orders could be tracked through matching, fill progression, cancellation, replacement, and rejection in a way that remained logically coherent and auditable.

At the same time, the scope was carefully limited to avoid overlap with other groups. Group 1 did not own market data ingestion, and it did not own the broader order book or downstream reporting layers beyond the terminal lifecycle record. Those components were outside the responsibility boundary by design. This separation allowed the team to focus on the correctness of the order lifecycle itself while still integrating cleanly with other parts of the system.

Screenshot placement: put one screenshot immediately after this subsection showing the complete order-flow chain in the code, so the reader can visually connect the ingress layer, orchestration layer, and persistence layer in one place.

3.2 Team Responsibility Split

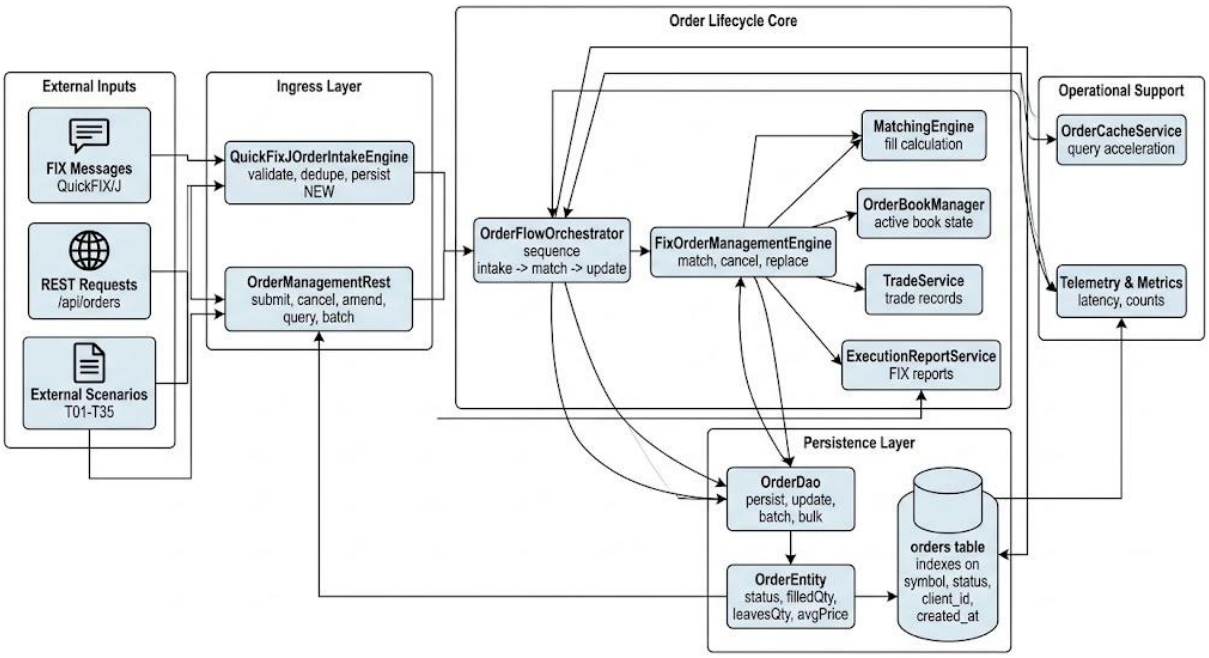
Within Group 1, the work was divided into six functional areas so that multiple members could work in parallel without creating confusion at the interfaces. The split was based on the structure of the order lifecycle itself, which made coordination easier because each member could own a clearly defined stage of the process while still contributing to a single integrated pipeline.

- The first area was FIX handling and execution-report plumbing. This role focused on translating Quick FIX/J messages into internal order events and ensuring that protocol-level messages were handled cleanly without leaking FIX-specific complexity into the rest of the application. In simple terms, this layer acted as the protocol adapter that turned incoming FIX messages into meaningful order actions.

- The second area was order pipeline orchestration. This responsibility covered the sequencing of the major processing steps: intake, validation, persistence, matching, and post-match updates. The orchestrator was the control layer that made sure an

accepted order moved through the system in the correct order and that both pre-match and post-match state changes were handled consistently.

- The third area was matching behaviour and contra fill integration. This work focused on what happens when orders cross, including how fill quantities are computed, how execution prices are determined, and how matching outcomes are reflected into the order lifecycle. This area was essential because matching is where the system produces business truth, and that truth must be propagated accurately into both storage and outward-facing reporting.
- The fourth area, and my primary responsibility, was persistence and database integration. This included the entity model, schema design, DAO implementation, indexing strategy, and batch writes. This role was central because every externally visible guarantee depends on durable storage reflecting the real lifecycle state of each order. Without this layer, the system could accept orders and even match them correctly in memory, but it would not have a reliable or query able record of what happened.
- The fifth area was REST lifecycle and control endpoints. This included order submission, amendment, bulk cancel, and query operations over HTTP. These endpoints were needed to support operational workflows, testing, and validation outside the FIX channel, and they had to follow the same lifecycle rules as the protocol-driven path.
- The sixth area was scenario reliability and regression support. This cross-cutting responsibility focused on running and stabilizing the external scenario suite, especially stress-heavy workflows where timing issues and race conditions were most likely to surface. This work was important because it proved that the lifecycle implementation held up under repeated end-to-end execution rather than only in isolated tests.



4. End to End Order Lifecycle Behaviour

4.1 Intake and Duplicate Rejection

The order lifecycle begins at the intake layer, where the FIX intake engine performs a duplicate-order check before any deeper processing takes place. This is important because a trading system can receive the same logical order more than once due to FIX session recovery, network retransmission, or client-side resubmission after an uncertain acknowledgement. By checking the persistence layer first, the engine ensures that duplicate detection is based on durable order history rather than transient in-memory state. If a record with the same client order ID already exists, the message is rejected immediately and does not enter the rest of the processing pipeline. This protects the system from accidental double acceptance and preserves order identity correctness at the boundary.

```
if (orderDao.findByClOrdId(clOrdId) != null) {
    telemetryService.recordFixMessageRejected();
    telemetryService.recordOrderRejected();
    return IntakeResult.rejected(clOrdId, symbol: "UNKNOWN", side: '1', quantity: 0, rea
}
```

4.2 Baseline State Initialisation and Early Persistence

Once an order passes duplicate screening, it is converted into a canonical internal order record and written to persistence with baseline lifecycle values. The order is initialised with status NEW, filled quantity set to zero, leaves quantity set to the full original quantity, and average price set to zero. This early write is architecturally important because it gives the system a durable record immediately upon acceptance, even before matching begins. In practical terms, this means the order can always be recovered, audited, or queried after intake, rather than existing only in temporary runtime memory. Early persistence also supports the uniqueness constraint at the database level, so the system has both application-side and storage-side protection against duplicate acceptance.

```
orderValidationService.enrichOrder(order);
order.setStatus(status: "NEW");
try {
    orderDao.persistOrder(order);
    orderCacheService.addToCache(order);
} catch (RuntimeException e) {
    processedClOrdIds.remove(clOrdId);
    throw e;
}
```

4.3 Orchestrated Persistence Before and After Matching

The order flow orchestrator uses a deliberate two-write lifecycle model. First, it persists the accepted order in baseline NEW state, and then after matching completes it writes back the execution outcome. This separation is intentional because it preserves both the accepted intent and the matched result as part of the lifecycle record. The first write proves the order was accepted; the second write records what happened to it during matching. That trade-off slightly increases database writes, but it gives a much clearer and more reliable audit trail,

especially when scenarios involve partial fills or cancellation timing issues. In a trading system, this is a better design than only storing the result, because intermediate lifecycle truth matters for debugging, reconciliation, and scenario validation.

```
order.setStatus(status: "NEW");
order.setFilledQty(filledQty: 0L);
order.setLeavesQty(order.getQuantity());
order.setAvgPrice(avgPrice: 0.0);
orderDao.persistOrder(order);
orderCacheService.addToCache(order);
performanceMetricsService.recordLatency(operation: "db.write", System.nanoTime() - stepStart);

auditTrailService.logOrderAccepted(clOrdId, order.getOrderRefNumber());

// 9. Match Order
stepStart = System.nanoTime();
OrderBookManager.BookOrder bookOrder = createBookOrder(order);
MatchingEngine.MatchResult matchResult = matchingEngine.matchOrder(bookOrder);
performanceMetricsService.recordLatency(operation: "order.match", System.nanoTime() - stepStart);

// 10. Update Order with Match Result
stepStart = System.nanoTime();
order.setStatus(matchResult.status.name());
order.setFilledQty((long) matchResult.filledQty);
order.setLeavesQty((long) matchResult.leavesQty);
order.setAvgPrice(matchResult.avgPrice);
orderDao.updateOrder(order);
```

4.4 Fill and Status Consistency

When the matching engine returns a fill result, the implementation updates the execution-related fields together, so the stored record remains mathematically and logically consistent. Filled quantity, leaves quantity, average price, and status are updated as a single lifecycle unit rather than as independent fields. The key invariant is that filled quantity plus leaves quantity must always equal the original order quantity, so leaves quantity is derived from the updated filled quantity instead of being adjusted separately. Status is then derived from the resulting leaves quantity: if no quantity remains, the order becomes FILLED; otherwise, it becomes PARTIALLY_FILLED. Average price is also updated using the actual fill data, ensuring that the stored execution price reflects the true weighted outcome of the match rather than only the latest trade.

```
if (result.filledQty > 0) {
    order.setFilledQty((long) result.filledQty);
    order.setLeavesQty((long) result.leavesQty);

    double totalValue = 0;
    for (MatchingEngine.Fill fill : result.fills) {
        totalValue += fill.price * fill.quantity;

        tradeService.createTrade(fill, order.getOrderRefNumber(), order.getClOrdId(),
                order.getClientId(), order.getSide(), order.getSymbol());

        executionReportService.processContraFill(fill, sessionID);
    }
    order.setAvgPrice(totalValue / result.filledQty);

    if (result.leavesQty == 0) {
        order.setStatus(status: "FILLED");
        if (telemetryService != null) {
            telemetryService.recordOrderFilled();
        }
    } else {
        order.setStatus(status: "PARTIALLY_FILLED");
    }
```

4.5 Cancel and Replace Lifecycle Safety

Cancel and replace are implemented as guarded state transitions rather than direct updates. When a cancel request arrives, the handler first checks the stored order status and rejects the request if the order is already FILLED, CANCELED, or REJECTED. Only orders that are still active are allowed to proceed to cancellation. This matters because a trading system must never treat terminal orders as if they are still open. Replace semantics follow the same principle: the original order is closed out, and a replacement is introduced with a new client order ID, so the lifecycle remains explicit and traceable rather than ambiguous. This design prevents illegal state transitions and ensures the active order set in storage matches the actual lifecycle state of the system.

```java
public CancelResult cancel(String origClOrdId) {
    OrderEntity order = orderDao.findByClOrdId(origClOrdId);
    if (order == null) {
        return CancelResult.notFound();
    }

    String status = order.getStatus();
    if ("FILLED".equals(status) || "CANCELED".equals(status) || "REJECTED".equals(status)) {
        return CancelResult.tooLate(order, "Order already " + status);
    }

    if ("PENDING_CANCEL".equals(status)) {
        return CancelResult.pending(order, reason: "Order already pending cancel");
    }

    orderBookManager.removeOrder(order.getSymbol(), order.getOrderRefNumber());
    order.setStatus(status: "CANCELED");
    order.setUpdatedAt(LocalDateTime.now());
    orderDao.updateOrder(order);
    return CancelResult.success(order);      You, 17 hours ago • feat: stabilize scenarios, clean repo, and reor…
}

public ReplaceResult replace(String origClOrdId, String newClOrdId, Double newQty, Double newPrice) {
    OrderEntity order = orderDao.findByClOrdId(origClOrdId);
    if (order == null) {
        return ReplaceResult.notFound();
    }
```

```java
String status = order.getStatus();
if ("FILLED".equals(status) || "CANCELED".equals(status) || "REJECTED".equals(status)) {
    return ReplaceResult.tooLate(order, "Order already " + status);
}

orderBookManager.removeOrder(order.getSymbol(), order.getOrderRefNumber());

if (newQty != null) {
    long qty = newQty.longValue();
    order.setQuantity(qty);
    long filled = order.getFilledQty() != null ? order.getFilledQty() : 0L;
    order.setLeavesQty(Math.max(a: 0L, qty - filled));
}

if (newPrice != null) {
    order.setPrice(newPrice);
}

order.setClOrdId(newClOrdId);
order.setStatus(status: "NEW");
order.setUpdatedAt(LocalDateTime.now());
orderDao.updateOrder(order);
return ReplaceResult.success(order);
```

5. Deep-Dive: G1-M4 - Persistence and Database Integration

5.1 Why Persistence Was the Core Risk Area

Persistence was not a secondary implementation detail in this project; it was the main risk area. The external scenario suite evaluates system behaviour from the outside, which means a system can appear correct if it returns the right API response while still storing an inconsistent internal record. That is a dangerous failure mode in a trading system because correctness is not only about what the user sees at one moment, but also about what the database records over time.

This becomes especially important in two situations. The first is restart or recovery, where the system must reload state from storage and continue from the last durable truth. If persistence is wrong, recovery will be wrong as well. The second is stress execution, where in-memory and database state can drift under high activity if writes are delayed, skipped, or performed in the wrong sequence. For this reason, lifecycle-consistent persistence was one of the most important engineering tasks in the project, and it had to be treated as part of the correctness model rather than as a low-level implementation detail.

```java
@Entity
@Table(name = "orders")
public class OrderEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String orderRefNumber;
    private String clOrdId;
    private String symbol;
    private String side;
    private Long quantity;
    private Double price;
    private String orderType;
    private String timeInForce;
    private String status;
    private String clientId;
    private Long filledQty;
    private Long leavesQty;
    private Double avgPrice;
```

```sql
-- Orders table
CREATE TABLE IF NOT EXISTS orders (
    id SERIAL PRIMARY KEY,
    order_ref_number VARCHAR(50) NOT NULL UNIQUE,
    cl_ord_id VARCHAR(50) NOT NULL UNIQUE,
    symbol VARCHAR(10) NOT NULL,
    side CHAR(1) NOT NULL,
    quantity BIGINT NOT NULL,
    price NUMERIC(15, 4),
    order_type VARCHAR(10),
    time_in_force VARCHAR(10),
    status VARCHAR(50),
    client_id VARCHAR(50),
    filled_qty BIGINT DEFAULT 0,
    leaves_qty BIGINT,
    avg_price NUMERIC(15, 4),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (symbol) REFERENCES security_master(symbol),
    FOREIGN KEY (client_id) REFERENCES customer_master(customer_id)
);
```

```java
// 8. Persist Order
stepStart = System.nanoTime();
order.setStatus(status: "NEW");
order.setFilledQty(filledQty: 0L);
order.setLeavesQty(order.getQuantity());
order.setAvgPrice(avgPrice: 0.0);
orderDao.persistOrder(order);
orderCacheService.addToCache(order);
performanceMetricsService.recordLatency(operation: "db.write", System.nanoTime() - stepStart);

auditTrailService.logOrderAccepted(clOrdId, order.getOrderRefNumber());

// 9. Match Order
stepStart = System.nanoTime();
OrderBookManager.BookOrder bookOrder = createBookOrder(order);
MatchingEngine.MatchResult matchResult = matchingEngine.matchOrder(bookOrder);
performanceMetricsService.recordLatency(operation: "order.match", System.nanoTime() - stepStart);

// 10. Update Order with Match Result
stepStart = System.nanoTime();
order.setStatus(matchResult.status.name());
order.setFilledQty((long) matchResult.filledQty);
order.setLeavesQty((long) matchResult.leavesQty);
order.setAvgPrice(matchResult.avgPrice);
orderDao.updateOrder(order);
orderCacheService.addToCache(order);
performanceMetricsService.recordLatency(operation: "db.write", System.nanoTime() - stepStart);
```

5.2 Entity-Level Modelling: The Order Entity Design

The Order Entity class is the JPA mapping of the orders table, and it was designed so that a single database row contains the meaningful lifecycle state of the order. This is a strong design choice because it means the record is self-describing. In practice, that means you can inspect one row and understand the current order status, how much has been filled, how much remains open, and what the average execution price is, without needing to join to another table or reconstruct state from event history.

The important lifecycle fields are status, filledQty, leavesQty, and avgPrice. These fields work together as a complete summary of the order's state. Status tells you where the order is in the lifecycle, filledQty tells you how much has already executed, leavesQty tells you how much remains open, and avgPrice gives the weighted execution cost of the fills that have

already happened. Because these values are stored directly in the entity, the system can maintain a clear and query able snapshot of the order at every step.

```
Aarav, 3 weeks ago | 1 author (Aarav)
@Entity
@Table(name = "orders")
public class OrderEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String orderRefNumber;
    private String clOrdId;
    private String symbol;
    private String side;
    private Long quantity;
    private Double price;
    private String orderType;
    private String timeInForce;
    private String status;
    private String clientId;
    private Long filledQty;
    private Long leavesQty;
    private Double avgPrice;

    // FIX session tracking for contra fill routing
    private String senderCompId;
    private String targetCompId;

    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
```

5.3 Schema-Level Design: The Orders Table

The database schema enforces correctness at the storage level, not just in application code. That is important because application-side checks can be bypassed by concurrency bugs, missed code paths, or future changes that forget to preserve the same rule. Database constraints act as the final line of defence and make the data model more trustworthy.

The orders table includes a primary key for internal identification, a unique client order ID for preventing duplicates, and lifecycle-related fields such as status, filled_qty, leaves_qty, and avg_price. These columns are not treated as optional metadata; they are first-class parts of the order record. This allows the database to directly represent open orders, partially filled orders, and completed orders in a form that is immediately usable for querying and validation.

5.4 Index Strategy: Design and Rationale

The index strategy was not chosen arbitrarily. It was based on the actual access patterns used by the REST layer and the scenario assertions. In a trading system, the most common reads are not random; they are usually based on symbol, status, client, or time. That means the best

index strategy is the one that directly supports those lookups rather than one that merely looks complete on paper.

Four single-column indexes were created for this reason. The symbol index supports instrument-based lookups and order-book style views. The status index supports active-order filtering, especially for NEW and PARTIALLY_FILLED records. The client_id index supports per-client retrieval and duplicate-check logic. The created_at index supports audit and temporal ordering queries. Together, these indexes improve the practical responsiveness of the system while keeping the design simple and understandable.

```sql
-- Create indexes for performance
CREATE INDEX idx_orders_symbol ON orders(symbol);
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_client_id ON orders(client_id);
CREATE INDEX idx_orders_created_at ON orders(created_at);
CREATE INDEX idx_trades_symbol ON trades(symbol);
CREATE INDEX idx_trades_executed_at ON trades(executed_at);
CREATE INDEX idx_event_log_order_ref ON event_log(order_ref_number);
CREATE INDEX idx_event_log_created_at ON event_log(created_at);
CREATE INDEX idx_messages_created_at ON messages(created_at);
CREATE INDEX idx_messages_direction ON messages(direction);
```

5.5 DAO Abstraction: Design and Method Surface

The DAO layer creates a clean boundary between business logic and persistence mechanics. This matters because domain services should not need to know whether the underlying storage is JPA, SQL, or some future persistence technology. By isolating persistence logic in a DAO, the order processing flow becomes easier to maintain, test, and extend.

The core method surface includes persist for insertion on acceptance, update for lifecycle changes after matching or cancel events, and findByClientOrderId for duplicate detection and cancel guarding. Beyond that, the DAO also provides batch and bulk operations that are useful during higher-load workflows. This design keeps the service layer focused on business rules while the DAO handles write mechanics and data access patterns.

```
Aarav, 2 weeks ago | 1 author (Aarav)
@ApplicationScoped        Aarav, last month • FIX Trading Simulator - Complete working implem…
public class OrderDao {

    private static final Logger LOG = LoggerFactory.getLogger(OrderDao.class.getName());

    @PersistenceContext
    EntityManager em;

    @Transactional
    public void persistOrder(OrderEntity order) {
        LOG.debug(format: "Persisting order: {}", order.getClOrdId());
        em.persist(order);
    }

    @Transactional
    public void updateOrder(OrderEntity order) {
        LOG.debug(format: "Updating order: {}", order.getClOrdId());
        em.merge(order);
    }
```

```
    @Transactional
    public int batchPersistOrders(List<OrderEntity> orders, int batchSize) {
        if (orders == null || orders.isEmpty()) {
            return 0;
        }

        LOG.info(format: "Batch persisting {} orders with batch size {}", orders.size(), batchSize);
        int count = 0;

        for (int i = 0; i < orders.size(); i++) {
            em.persist(orders.get(i));
            count++;

            // Flush and clear every batchSize inserts
            if ((i + 1) % batchSize == 0) {
                em.flush();
                em.clear();
                LOG.debug(format: "Flushed and cleared at index {}", i + 1);
            }
        }

        // Final flush for remaining entities        Aarav, last month • Presentation Ready: Complete FIX Trad
        em.flush();
        em.clear();

        LOG.info(format: "Successfully batch persisted {} orders", count);
        return count;
    }
```

```java
        LOG.info(format: "Successfully batch persisted {} orders", count);
        return count;
    }

    /**
     * Batch insert with default batch size of 50
     */
    @Transactional
    public int batchPersistOrders(List<OrderEntity> orders) {
        return batchPersistOrders(orders, batchSize: 50);
    }

    /**
     * Batch update orders for mass status changes
     */
    @Transactional
    public int batchUpdateOrders(List<OrderEntity> orders, int batchSize) {
        if (orders == null || orders.isEmpty()) {
            return 0;
        }

        LOG.info(format: "Batch updating {} orders with batch size {}", orders.size(), batchSize);
        int count = 0;

        for (int i = 0; i < orders.size(); i++) {
            em.merge(orders.get(i));
            count++;

            if ((i + 1) % batchSize == 0) {
                em.flush();
                em.clear();
            }
        }
```

```java
        em.flush();
        em.clear();

        LOG.info(format: "Successfully batch updated {} orders", count);
        return count;
    }

    /**
     * Bulk update status by IDs (efficient for cancellation workflows)
     */
    @Transactional
    public int bulkUpdateStatus(List<Long> orderIds, String newStatus) {
        if (orderIds == null || orderIds.isEmpty()) {
            return 0;
        }

        LOG.info(format: "Bulk updating status to {} for {} orders", newStatus, orderIds.size());

        int updated = em.createQuery(
            qlString: "UPDATE OrderEntity o SET o.status = :status WHERE o.id IN :ids")
            .setParameter(name: "status", newStatus)
            .setParameter(name: "ids", orderIds)
            .executeUpdate();

        LOG.info(format: "Bulk updated {} orders to status {}", updated, newStatus);
        return updated;
    }
```

5.6 Batch Throughput Design and Implementation

Batch operations become important as soon as the system starts receiving burst traffic. In scenarios like T30 and T33, the system is not handling a single isolated order at a time; it is processing many related actions in a short window. If every insert or update is done individually, database roundtrips accumulate quickly and become the main throughput bottleneck.

The batch persist method reduces that overhead by grouping writes into chunks, with a default size of 50. Instead of making one database call per order, the system writes a group of orders together, which significantly reduces latency and persistence-context pressure. The bulk status update method serves a similar role for operations like bulk cancel, where many orders need the same status transition. This is a practical throughput design because it improves performance without sacrificing the clarity of the lifecycle model.

5.7 Lifecycle Consistency via Orchestrator Writes

The orchestrator is responsible for making the persistence model match the order lifecycle model. It does this through explicit writes at distinct points in the flow. First, it stores the accepted order in NEW state with its baseline quantities. Then, after matching completes, it updates the row again with fill results and final status. Finally, cancel handling writes the cancelled state when the order is no longer active.

This two-step write structure is important because it preserves both the accepted intent and the eventual outcome. In a trading system, those are not the same thing, and both matter. The accepted order shows what entered the system, while the post-match update shows what happened to it. This is also why the cancel path is written carefully and atomically: if the status check and the state write were separated too loosely, the system could become inconsistent during concurrent activity.

```java
// 8. Persist Order
stepStart = System.nanoTime();
order.setStatus(status: "NEW");        Aarav, 3 weeks ago • Complete FIX Trading Simulator with enhanced UI…
order.setFilledQty(filledQty: 0L);
order.setLeavesQty(order.getQuantity());
order.setAvgPrice(avgPrice: 0.0);
orderDao.persistOrder(order);
orderCacheService.addToCache(order);
performanceMetricsService.recordLatency(operation: "db.write", System.nanoTime() - stepStart);

auditTrailService.logOrderAccepted(clOrdId, order.getOrderRefNumber());

// 9. Match Order
stepStart = System.nanoTime();
OrderBookManager.BookOrder bookOrder = createBookOrder(order);
MatchingEngine.MatchResult matchResult = matchingEngine.matchOrder(bookOrder);
performanceMetricsService.recordLatency(operation: "order.match", System.nanoTime() - stepStart);

// 10. Update Order with Match Result
stepStart = System.nanoTime();
order.setStatus(matchResult.status.name());
order.setFilledQty((long) matchResult.filledQty);
order.setLeavesQty((long) matchResult.leavesQty);
order.setAvgPrice(matchResult.avgPrice);
orderDao.updateOrder(order);
orderCacheService.addToCache(order);
performanceMetricsService.recordLatency(operation: "db.write", System.nanoTime() - stepStart);
```

6. FIX Path, REST Path, and Integration Cohesion

6.1 The FIX Protocol Path

Quick FIX/J acts as the FIX engine and handles session-level concerns such as message parsing, sequencing, and protocol transport. The role of Exchange Application is to connect those incoming FIX events to the application logic that manages order lifecycle state. This separation is important because protocol handling and business logic are not the same thing, and they should not be mixed.

In the FIX path, New Order Single messages are routed to the intake engine for deduplication and initial persistence. Cancel requests are routed to the cancel logic, and replace requests are routed to the replace logic. In all three cases, the FIX layer handles the protocol format, but the actual order decisions are made by the engine and orchestrator layers. Execution reports are sent only after the write path confirms the new state, which keeps the client's view aligned with the stored order record.

Reference-ExchangeApplication.java:87-443

6.2 The REST API Path and Batch Telemetry

The REST API was designed to use the same business logic as the FIX path rather than duplicating its own version of the workflow. This is a strong architectural decision because it keeps the system consistent regardless of whether an order arrives through FIX or through HTTP. It also reduces maintenance burden, because there is one real order-processing pipeline instead of multiple diverging ones.

The REST interface supports order submission, batch submit, amend, and bulk cancel. The batch submit endpoint is especially useful because it exposes telemetry such as throughput and latency, which help measure how the system behaves under load. Those metrics are valuable during stress validation because they show where the pipeline is slowing down and whether batch-style operations are performing as expected. In practice, the REST path is both a functional interface and an operational testing surface.

```java
@POST
@Path("/{orderRefNumber}/amend")
public Response amendOrder(
        @PathParam("orderRefNumber") String orderRefNumber,
        AmendOrderRequest request) {

    if (request == null) {
        return Response.status(Response.Status.BAD_REQUEST)
                .entity(Map.of(k1: "error", v1: "Amend request body is required")).build();      Aarav, 2 weeks ago • Update t
    }

    if (request.newPrice < 0 || request.newQuantity < 0) {
        return Response.status(Response.Status.BAD_REQUEST)
                .entity(Map.of(k1: "error", v1: "newPrice and newQuantity must be non-negative")).build();
    }

    if (request.newPrice == 0 && request.newQuantity == 0) {
        return Response.status(Response.Status.BAD_REQUEST)
                .entity(Map.of(k1: "error", v1: "Provide at least one field to amend: newPrice or newQuantity")).build();
    }

    // Find the original order
    OrderEntity original = orderDao.findByOrderRefNumber(orderRefNumber);
    if (original == null) {
        return Response.status(Response.Status.NOT_FOUND)
                .entity(Map.of(k1: "error", "Order not found: " + orderRefNumber)).build();
    }

    // Can only amend NEW or PARTIALLY_FILLED orders
    String status = original.getStatus();
    if (!"NEW".equals(status) && !"PARTIALLY_FILLED".equals(status)
        && !"PARTIAL_FILL".equals(status) && !"PENDING_NEW".equals(status)) {
        return Response.status(Response.Status.BAD_REQUEST)
                .entity(Map.of(k1: "error", "Cannot amend order in status: " + status)).build();
    }

    // Cancel original
    String reason = "Replaced by amendment";
    String clientId = request.clientId != null ? request.clientId : original.getClientId();
    OrderCancelService.CancelResult cancelResult = cancelService.requestCancel(orderRefNumber, clientId, reason);
    if (!cancelResult.success && !"ALREADY_CANCELED".equals(cancelResult.message)) {
        return Response.status(Response.Status.CONFLICT)
                .entity(Map.of(k1: "error", "Failed to cancel original order: " + cancelResult.message)).build();
    }

    // Build replacement request
    OrderSubmitRequest replacement = new OrderSubmitRequest();
    replacement.symbol = original.getSymbol();
    replacement.side = original.getSide();
    replacement.quantity = request.newQuantity > 0 ? request.newQuantity : original.getQuantity().intValue();
    replacement.price = request.newPrice > 0 ? request.newPrice : original.getPrice();
    replacement.orderType = original.getOrderType();
    replacement.timeInForce = original.getTimeInForce();
    replacement.clientId = clientId;
    replacement.clOrdId = "AMD-" + orderRefNumber.substring(Math.max(a: 0, orderRefNumber.length() - 8));

    // Submit replacement via orchestrator
    Response submitResponse = submitOrchestratedOrder(replacement);

    if (submitResponse.getStatus() == 201) {
        @SuppressWarnings("unchecked")
        Map<String, Object> body = (Map<String, Object>) submitResponse.getEntity();
        body.put(key: "amendedFrom", orderRefNumber);
        body.put(key: "priceChanged", request.newPrice > 0);
        body.put(key: "quantityChanged", request.newQuantity > 0);
        return Response.ok(body).build();
    }
```

```java
/**
 * Bulk cancel - cancel all orders for a client or symbol
 */
@POST
@Path("/bulk-cancel")
public Response bulkCancel(BulkCancelRequest request) {
    if (request == null) {
        request = new BulkCancelRequest();
    }

    String requestClientId = request.clientId != null && !request.clientId.isBlank()
        ? request.clientId
        : "CLIENT001";
    String reason = request.reason != null && !request.reason.isBlank()
        ? request.reason
        : "Bulk cancel";

    int canceledCount = 0;
    int failedCount = 0;
    java.util.List<String> errors = new java.util.ArrayList<>();

    java.util.List<com.helesto.model.OrderEntity> orders;
    if (request.symbol != null && !request.symbol.isEmpty()) {
        orders = orderDao.findBySymbol(request.symbol);
    } else if (request.clientId != null && !request.clientId.isEmpty()) {
        orders = orderDao.findByClientId(request.clientId);
    } else {
        // Default bulk cleanup should target all active states, not only NEW.
        java.util.LinkedHashMap<String, com.helesto.model.OrderEntity> byRef = new java.util.LinkedHashMap<>();
        for (com.helesto.model.OrderEntity o : orderDao.findByStatus(status: "NEW")) {
            byRef.put(o.getOrderRefNumber(), o);
```

```duplicate_block_1
    } else {
        // Default bulk cleanup should target all active states, not only NEW.
        java.util.LinkedHashMap<String, com.helesto.model.OrderEntity> byRef = new java.util.LinkedHashMap<>();
        for (com.helesto.model.OrderEntity o : orderDao.findByStatus(status: "NEW")) {
            byRef.put(o.getOrderRefNumber(), o);
        }
```

```java
        for (com.helesto.model.OrderEntity o : orderDao.findByStatus(status: "PARTIALLY_FILLED")) {
            byRef.put(o.getOrderRefNumber(), o);
        }
        for (com.helesto.model.OrderEntity o : orderDao.findByStatus(status: "PARTIAL_FILL")) {
            byRef.put(o.getOrderRefNumber(), o);
        }
        orders = new java.util.ArrayList<>(byRef.values());
    }

    for (com.helesto.model.OrderEntity o : orders) {
        if ("NEW".equals(o.getStatus()) || "PARTIALLY_FILLED".equals(o.getStatus())
            || "PARTIAL_FILL".equals(o.getStatus())) {
            String effectiveClientId = (o.getClientId() != null && !o.getClientId().isBlank())
                ? o.getClientId()
                : requestClientId;
            OrderCancelService.CancelResult r = cancelService.requestCancel(
                o.getOrderRefNumber(), effectiveClientId, reason);
            if (r.success) canceledCount++;
            else { failedCount++; errors.add(o.getOrderRefNumber() + ": " + r.message); }
        }
    }
```

7. Testing and Validation Strategy

7.1 Validation Philosophy and Test Design

The testing approach for this project was designed around lifecycle correctness, not only endpoint correctness. In other words, a test was considered successful only when an order moved through the full state path correctly and the resulting persistent record matched observable behaviour. This prevented shallow pass conditions where API responses looked correct while stored state drifted over time. The team intentionally combined protocol-driven and API-driven test execution so both ingress paths validated the same lifecycle logic and produced comparable outcomes. This testing philosophy was critical for identifying hidden sequencing bugs and state drift that would not appear in isolated unit tests.

7.2 External Scenario Suite as Primary Acceptance Gate

The external Mini FIX scenario suite served as the primary acceptance mechanism for feature closure and regression confidence. Instead of validating one script once, the team repeatedly executed the full suite and required stable behaviour across all scenario groups before considering a change complete. This gave stronger confidence than ad hoc local checks because the suite covered basic workflows, cancel/modify paths, edge cases, session handling, and stress conditions in one integrated run. The run summary mechanism in the suite runner made this process auditable by reporting total, passed, and failed counts for each regression cycle.

```
$results = @()
$index = 1

foreach ($cat in $categoryDirs) {
    $tests = Get-ChildItem -Path $cat.FullName -File -Filter 'T*.txt' | Sort-Object Name
    foreach ($test in $tests) {
        Remove-Item $markerPath -Force -ErrorAction SilentlyContinue

        try {
            Invoke-RestMethod -Method Post -Uri ("http://{0}:{1}/api/orders/bulk-cancel" -f $OMS_HOST, $cleanupPort) -ContentType '
        } catch {
            Write-Host ("WARN: cleanup failed before {0}: {1}" -f $test.Name, $_.Exception.Message) -ForegroundColor Yellow
        }

        $sw = [System.Diagnostics.Stopwatch]::StartNew()
        & "$suiteDir\run_single.bat" $test.FullName      You, 18 hours ago • feat: stabilize scenarios, clean repo, and reor…
        $code = $LASTEXITCODE
        $sw.Stop()

        $status = if ($code -eq 0) { 'PASS' } else { 'FAIL' }
        $statusColor = if ($status -eq 'PASS') { 'Green' } else { 'Red' }
        Write-Host ("[{0}] {1} ({2:n1}s)" -f $status, $test.Name, $sw.Elapsed.TotalSeconds) -ForegroundColor $statusColor

        $results += [PSCustomObject]@{
            Num      = $index
            Category = $cat.Name
            Test     = $test.Name
            Status   = $status
            Time     = ('{0:n1}s' -f $sw.Elapsed.TotalSeconds)
        }
```

```
        }

        $index++
        Start-Sleep -Milliseconds 200
    }
}

$total = $results.Count
$passed = ($results | Where-Object { $_.Status -eq 'PASS' }).Count
$failed = ($results | Where-Object { $_.Status -eq 'FAIL' }).Count

Write-Host ''
Write-Host ("Total: {0} | Passed: {1} | Failed: {2}" -f $total, $passed, $failed) -ForegroundColor Cyan
$results | Format-Table @{Label='#';Expression={$_.Num}}, Category, Test, @{Label='Result';Expression={$_.Status}}, Time -AutoSize

if ($failed -gt 0) {
    Write-Host ''
    Write-Host 'Failed tests:' -ForegroundColor Red
    $results | Where-Object { $_.Status -eq 'FAIL' } | ForEach-Object { Write-Host (" - {0}\{1}" -f $_.Category, $_.Test) -ForegroundColor Red
}

Write-Host ''
Write-Host 'Tip: Run individual tests for detailed failure output: .\run_single.bat <path\to\test.txt>' -ForegroundColor Yellow

Get-Process MiniFIX -ErrorAction SilentlyContinue | Stop-Process -Force -ErrorAction SilentlyContinue
```

7.3 Stress Validation and Race-Prone Scenario Hardening

Stress validation focused on scenarios where lifecycle consistency is most likely to break under pressure, especially rapid submit/cancel and mixed-operation workloads. The T30 to T35 group was treated as high-risk because it compresses many state transitions into short execution windows, increasing the chance of timing-related inconsistencies between in-memory and persisted state. T33 was especially important because cancel storm behaviour can expose weak transition guards and delayed persistence writes. Stabilising this class of scenario required ensuring that lifecycle updates stayed deterministic even when operations arrived rapidly.

```
# T33 - Cancel Storm (100 Submit-Cancel Pairs)
# ==============================================================================
# Validates: OMS handles rapid mass cancellation. 100 orders are submitted
#            and immediately canceled as fast as possible.
# Symbol: MSFT (all buys at random prices so nothing auto-matches)
# Measures: Cancellation throughput
# ==============================================================================

Reset
SenderCompId:{{SENDER_COMP_ID}}
TargetCompId:{{TARGET_COMP_ID}}
Connect:{{OMS_HOST}}:{{OMS_PORT}}

# Expect Logon acknowledgment
Wait:35=A

Message:T33 SUBMIT_CANCEL_START $TIMESTAMP

# Submit + immediately cancel, 100 iterations (100 orders, 100 cancels)      You, 18 hours ago • feat: stabilize scenarios, clean r
Execute:helpers\T33_submit_cancel.txt:100

Message:T33 SUBMIT_CANCEL_END $TIMESTAMP

# Allow responses to settle
Sleep:5000

Message:T33 PASSED - 100 orders submitted and canceled

# Verify: most/new all new-order acks are received
VerifyCount:35=8|150=0:>=90
# Verify: at least some submitted orders complete through cancel execution reports.
# Under heavy async races, rejects and cancel-exec distributions can vary widely by run.
VerifyCount:35=8|150=4:>=1

Save:results\T33_result.txt
```

7.4 Engine-Level Verification and Targeted Unit Coverage

Alongside external scenarios, targeted service-level tests were used to verify critical transition logic such as cancel eligibility, replace semantics, and state updates for guarded paths. This layer of testing gave faster feedback for high-impact behaviours before running full regression, and it reduced debugging time by isolating whether a failure was local to business logic or systemic across integrated flow. The combination of focused engine tests and full-suite execution created a practical balance between speed and confidence.

```java
class FixOrderManagementEngineTest {

    @Test
    void cancelShouldMarkOrderCanceledAndRemoveFromBook() {
        InMemoryOrderDao dao = new InMemoryOrderDao();
        OrderBookManager bookManager = new OrderBookManager();

        OrderEntity order = new OrderEntity();
        order.setClOrdId(clOrdId: "C1");
        order.setOrderRefNumber(orderRefNumber: "R1");
        order.setSymbol(symbol: "AAPL");
        order.setSide(side: "1");        You, 18 hours ago • feat: stabilize scenarios, clean repo, and reor…
        order.setQuantity(quantity: 100L);
        order.setFilledQty(filledQty: 0L);
        order.setLeavesQty(leavesQty: 100L);
        order.setStatus(status: "NEW");
        dao.put(order);

        OrderBookManager.BookOrder bookOrder = new OrderBookManager.BookOrder();
        bookOrder.orderId = "R1";
        bookOrder.clOrdId = "C1";
        bookOrder.symbol = "AAPL";
        bookOrder.side = "1";
        bookOrder.price = 150.0;
        bookOrder.originalQty = 100;
        bookOrder.leavesQty = 100;
        bookOrder.orderType = "LIMIT";
        bookOrder.timeInForce = "DAY";
        bookManager.addOrder(bookOrder);

        FixOrderManagementEngine engine = new FixOrderManagementEngine();
        setField(engine, fieldName: "orderDao", dao);
        setField(engine, fieldName: "orderBookManager", bookManager);
```

```java
        Assertions.assertTrue(result.success);
        Assertions.assertEquals("C2", result.order.getClOrdId());
        Assertions.assertEquals(120L, result.order.getQuantity());
        Assertions.assertEquals(100L, result.order.getLeavesQty());
        Assertions.assertEquals(151.0, result.order.getPrice());
        Assertions.assertEquals("NEW", result.order.getStatus());
    }

    @Test
    void cancelShouldRejectAlreadyFilledOrder() {
        InMemoryOrderDao dao = new InMemoryOrderDao();

        OrderEntity order = new OrderEntity();
        order.setClOrdId(clOrdId: "C1");
        order.setOrderRefNumber(orderRefNumber: "R1");
        order.setSymbol(symbol: "AAPL");
        order.setStatus(status: "FILLED");
        dao.put(order);

        FixOrderManagementEngine engine = new FixOrderManagementEngine();
        setField(engine, fieldName: "orderDao", dao);
        setField(engine, fieldName: "orderBookManager", new OrderBookManager());

        FixOrderManagementEngine.CancelResult result = engine.cancel(origClOrdId: "C1");

        Assertions.assertFalse(result.success);
        Assertions.assertTrue(result.tooLate);
    }
    FixOrderManagementEngine.CancelResult result = engine.cancel(origClOrdId: "C1");

    Assertions.assertTrue(result.success);
    Assertions.assertEquals("CANCELED", dao.findByClOrdId(clOrdId: "C1").getStatus());
    Assertions.assertTrue(bookManager.getBidsAtPrice(symbol: "AAPL", price: 150.0).isEmpty());
}

@Test
void replaceShouldApplyNewValuesAndKeepLeavesConsistent() {
    InMemoryOrderDao dao = new InMemoryOrderDao();

    OrderEntity order = new OrderEntity();
    order.setClOrdId(clOrdId: "C1");
    order.setOrderRefNumber(orderRefNumber: "R1");
    order.setSymbol(symbol: "AAPL");
    order.setSide(side: "1");
    order.setQuantity(quantity: 100L);
    order.setFilledQty(filledQty: 20L);
    order.setLeavesQty(leavesQty: 80L);
    order.setPrice(price: 150.0);
    order.setStatus(status: "PARTIALLY_FILLED");
    dao.put(order);

    FixOrderManagementEngine engine = new FixOrderManagementEngine();
    setField(engine, fieldName: "orderDao", dao);
    setField(engine, fieldName: "orderBookManager", new OrderBookManager());

    FixOrderManagementEngine.ReplaceResult result = engine.replace(origClOrdId: "C1", newClOrdId: "C2", newQty: 120.0, newPrice:
```

8. Design Rationale and Trade-Off Analysis

8.1 Two-Step Persistence Model

The most important persistence decision in this system is the two-step write model: one writes when the order is accepted, and another after matching completes. This design ensures that acceptance is recorded as a durable business event, not just an in-memory action. If the process crashes between intake and matching, the system still has a persisted acceptance record and can recover consistently.
The trade-off is additional write volume. However, this cost is acceptable because it provides stronger lifecycle traceability and prevents a high-impact failure mode where a client receives acceptance semantics, but storage contains no corresponding order record. In burst conditions, batch-oriented DAO operations reduce the practical cost of this pattern, making the additional write overhead manageable.

8.2 Consistent Multi-Field Lifecycle Updates

After matching, lifecycle-critical fields are updated together in one persistence path: filled quantity, leaves quantity, average price, and status. Treating these values as a single unit prevents logical drift, such as a terminal status with non-zero leaves or a partial status with inconsistent quantities. The design objective is that stored state always reflects a coherent snapshot of execution progress.
This approach also improves auditability. When all execution fields move together, downstream query consumers and scenario assertions read a stable lifecycle view without needing to infer or repair partial state.

8.3 DAO as the Exclusive Persistence Boundary

Using the DAO layer as the single persistence boundary keeps business logic and storage logic cleanly separated. Orchestrator and engine services define lifecycle rules; DAO methods execute persistence mechanics. This separation improves maintainability, reduces duplication of query logic, and makes the codebase easier to test in isolation.
The DAO boundary is also where performance-aware methods are concentrated, including batch inserts, batch updates, and bulk status updates. Centralizing these operations prevents persistence behaviour from becoming fragmented across services.

8.4 Indexing Based on Real Query Behaviour

Index selection was based on observed query access paths in operational APIs and scenario workflows, not speculative future assumptions. The chosen indexes support symbol, status, client, and time-based retrieval because those are the actual dimensions repeatedly used for monitoring, validation, and operational actions.
The trade-off is that future query shapes may require additional indexes later. This is acceptable and often preferable, because reactive indexing based on real workloads avoids unnecessary write amplification from speculative index creation.

9. Validation Evidence and Quality Signals

9.1 Unit-Level Evidence

Unit validation was used to confirm that lifecycle guard logic behaves correctly at service level before running full end-to-end flows. The tests around order management verify that active orders can be cancelled correctly, replace operations apply amended values without breaking lifecycle semantics, and terminal-state orders are protected from invalid cancellation. This test layer is important because it proves the transition rules directly, not just indirect side effects from larger scenario runs. In addition, deterministic unit tests reduce debugging time by isolating logic regressions quickly before they propagate into integration pipelines.

```java
class FixOrderManagementEngineTest {

    @Test
    void cancelShouldMarkOrderCanceledAndRemoveFromBook() {
        InMemoryOrderDao dao = new InMemoryOrderDao();
        OrderBookManager bookManager = new OrderBookManager();

        OrderEntity order = new OrderEntity();
        order.setClOrdId(clOrdId: "C1");
        order.setOrderRefNumber(orderRefNumber: "R1");
        order.setSymbol(symbol: "AAPL");
        order.setSide(side: "1");        You, 18 hours ago • feat: stabilize scenarios, clean repo, and
        order.setQuantity(quantity: 100L);
        order.setFilledQty(filledQty: 0L);
        order.setLeavesQty(leavesQty: 100L);
        order.setStatus(status: "NEW");
        dao.put(order);

        OrderBookManager.BookOrder bookOrder = new OrderBookManager.BookOrder();
        bookOrder.orderId = "R1";
        bookOrder.clOrdId = "C1";
        bookOrder.symbol = "AAPL";
        bookOrder.side = "1";
        bookOrder.price = 150.0;
        bookOrder.originalQty = 100;
        bookOrder.leavesQty = 100;
        bookOrder.orderType = "LIMIT";
        bookOrder.timeInForce = "DAY";
        bookManager.addOrder(bookOrder);

        FixOrderManagementEngine engine = new FixOrderManagementEngine();
        setField(engine, fieldName: "orderDao", dao);
        setField(engine, fieldName: "orderBookManager", bookManager);
```

```java
    FixOrderManagementEngine.CancelResult result = engine.cancel(origClOrdId: "C1");

    Assertions.assertTrue(result.success);
    Assertions.assertEquals("CANCELED", dao.findByClOrdId(clOrdId: "C1").getStatus());
    Assertions.assertTrue(bookManager.getBidsAtPrice(symbol: "AAPL", price: 150.0).isEmpty());
}

@Test
void replaceShouldApplyNewValuesAndKeepLeavesConsistent() {
    InMemoryOrderDao dao = new InMemoryOrderDao();

    OrderEntity order = new OrderEntity();
    order.setClOrdId(clOrdId: "C1");
    order.setOrderRefNumber(orderRefNumber: "R1");
    order.setSymbol(symbol: "AAPL");
    order.setSide(side: "1");
    order.setQuantity(quantity: 100L);
    order.setFilledQty(filledQty: 20L);
    order.setLeavesQty(leavesQty: 80L);
    order.setPrice(price: 150.0);
    order.setStatus(status: "PARTIALLY_FILLED");
    dao.put(order);

    FixOrderManagementEngine engine = new FixOrderManagementEngine();
    setField(engine, fieldName: "orderDao", dao);
    setField(engine, fieldName: "orderBookManager", new OrderBookManager());

    FixOrderManagementEngine.ReplaceResult result = engine.replace(origClOrdId: "C1", newClOrdId: "C2", newQty: 120.0, newPrice:

    Assertions.assertTrue(result.success);
    Assertions.assertEquals("C2", result.order.getClOrdId());
    Assertions.assertEquals(120L, result.order.getQuantity());
    Assertions.assertEquals(100L, result.order.getLeavesQty());
    Assertions.assertEquals(151.0, result.order.getPrice());
```

```java
    Assertions.assertEquals("NEW", result.order.getStatus());
}

@Test
void cancelShouldRejectAlreadyFilledOrder() {
    InMemoryOrderDao dao = new InMemoryOrderDao();

    OrderEntity order = new OrderEntity();
    order.setClOrdId(clOrdId: "C1");
    order.setOrderRefNumber(orderRefNumber: "R1");
    order.setSymbol(symbol: "AAPL");
    order.setStatus(status: "FILLED");
    dao.put(order);

    FixOrderManagementEngine engine = new FixOrderManagementEngine();
    setField(engine, fieldName: "orderDao", dao);
    setField(engine, fieldName: "orderBookManager", new OrderBookManager());

    FixOrderManagementEngine.CancelResult result = engine.cancel(origClOrdId: "C1");

    Assertions.assertFalse(result.success);
    Assertions.assertTrue(result.tooLate);
}
```

9.2 Integration and Stress Evidence

Integration validation provides the strongest correctness signal because it evaluates full lifecycle behaviour across ingress, orchestration, matching, persistence, and control paths. The external suite covers normal workflows, edge cases, session behaviour, and high-pressure stress runs, which makes it suitable for detecting timing-sensitive inconsistencies. Stress scenarios are especially important because they reveal failures that often remain hidden

in sequential tests. The T33 cancel-storm result is a high-value indicator, since it validates rapid submit/cancel cycles and confirms that lifecycle writes remain stable under load.

```
$total = $results.Count
$passed = ($results | Where-Object { $_.Status -eq 'PASS' }).Count
$failed = ($results | Where-Object { $_.Status -eq 'FAIL' }).Count

Write-Host ''
Write-Host ("Total: {0} | Passed: {1} | Failed: {2}" -f $total, $passed, $failed) -ForegroundColor Cyan
$results | Format-Table @{Label='#';Expression={$_.Num}}, Category, Test, @{Label='Result';Expression={$_.Status}}, Time -AutoSize

if ($failed -gt 0) {
    Write-Host ''
    Write-Host 'Failed tests:' -ForegroundColor Red
    $results | Where-Object { $_.Status -eq 'FAIL' } | ForEach-Object { Write-Host (" - {0}\{1}" -f $_.Category, $_.Test) -Foregrou
}

Write-Host ''
Write-Host 'Tip: Run individual tests for detailed failure output: .\run_single.bat <path\to\test.txt>' -ForegroundColor Yellow

Get-Process MiniFIX -ErrorAction SilentlyContinue | Stop-Process -Force -ErrorAction SilentlyContinue

if ($failed -gt 0) { exit 1 }        You, 18 hours ago • feat: stabilize scenarios, clean repo, and reor…
exit 0
```

10. Individual Contribution - Aarav Ashutosh Joshi (G1-M4)

10.1 Core Technical Ownership: Persistence and DB Integration

My primary ownership was the persistence layer and its lifecycle integrity. This included designing the order state model so that a single persisted record can represent status progression, matched quantity progression, remaining open quantity, and execution price evolution. I also owned schema-level decisions and DAO boundaries that keep business services independent of persistence implementation details. This architecture made order state durable, query able, and recoverable, which is essential for both correctness and operational confidence.

```
Aarav, 3 weeks ago | 1 author (Aarav)
@Entity
@Table(name = "orders")
public class OrderEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String orderRefNumber;
    private String clOrdId;
    private String symbol;
    private String side;
    private Long quantity;
    private Double price;
    private String orderType;
    private String timeInForce;
    private String status;
    private String clientId;
    private Long filledQty;
    private Long leavesQty;
    private Double avgPrice;
```

10.2 Throughput and Maintainability Contributions

Beyond correctness, I implemented persistence access patterns that support high-activity workflows without tightly coupling domain services to query logic. The DAO method surface includes standard lifecycle operations plus batch and bulk paths for larger write workloads. This improved maintainability, reduced duplication, and made stress behaviour more controllable by concentrating write semantics in one layer rather than spreading them across orchestration code.

```java
@ApplicationScoped        Aarav, last month • FIX Trading Simulator - Complete working implem…
public class OrderDao {

    private static final Logger LOG = LoggerFactory.getLogger(OrderDao.class.getName());

    @PersistenceContext
    EntityManager em;

    @Transactional
    public void persistOrder(OrderEntity order) {
        LOG.debug(format: "Persisting order: {}", order.getClOrdId());
        em.persist(order);
    }

    @Transactional
    public void updateOrder(OrderEntity order) {
        LOG.debug(format: "Updating order: {}", order.getClOrdId());
        em.merge(order);
    }

    @Transactional
    public OrderEntity findByClOrdId(String clOrdId) {
        LOG.info(format: "Finding order by ClOrdId: {}", clOrdId);
        return em.createQuery(qlString: "SELECT o FROM OrderEntity o WHERE o.clOrdId = :clOrdId", resultClass: OrderEnt
                .setParameter(name: "clOrdId", clOrdId)
                .getResultStream()
                .findFirst()
                .orElse(other: null);
    }

    @Transactional
    public OrderEntity findByOrderRefNumber(String orderRefNumber) {
        LOG.info(format: "Finding order by OrderRefNumber: {}", orderRefNumber);
        return em.createQuery(qlString: "SELECT o FROM OrderEntity o WHERE o.orderRefNumber = :orderRefNumber", resultC
                .setParameter(name: "orderRefNumber", orderRefNumber)
```

```java
                .getResultStream()
                .findFirst()
                .orElse(other: null);
    }

    public List<OrderEntity> findAll() {
        LOG.debug(msg: "Finding all orders");
        return em.createQuery(qlString: "SELECT o FROM OrderEntity o ORDER BY o.createdAt DESC", resultClass: OrderEntity.class)
                .getResultList();
    }

    public List<OrderEntity> findBySymbol(String symbol) {
        LOG.debug(format: "Finding orders by symbol: {}", symbol);
        return em.createQuery(qlString: "SELECT o FROM OrderEntity o WHERE o.symbol = :symbol ORDER BY o.createdAt DESC", resultClas
                .setParameter(name: "symbol", symbol)
                .getResultList();
    }

    public List<OrderEntity> findByStatus(String status) {
        LOG.debug(format: "Finding orders by status: {}", status);
        return em.createQuery(qlString: "SELECT o FROM OrderEntity o WHERE o.status = :status ORDER BY o.createdAt DESC", resultClas
                .setParameter(name: "status", status)
                .getResultList();
    }

    public List<OrderEntity> findByClientId(String clientId) {
        LOG.debug(format: "Finding orders by clientId: {}", clientId);
        return em.createQuery(qlString: "SELECT o FROM OrderEntity o WHERE o.clientId = :clientId ORDER BY o.createdAt DESC", result
                .setParameter(name: "clientId", clientId)
                .getResultList();
    }

    @Transactional
    public void update(OrderEntity order) {
```

```java
// ================= G1-M4: Batch Operations =================

/**
 * G1-M4: Batch insert orders for high-volume scenarios
 * - Flush and clear in configurable batches
 * - Optimal for bulk order submission
 *
 * @param orders List of orders to persist
 * @param batchSize Number of inserts before flush/clear
 * @return Number of orders successfully persisted
 */
@Transactional
public int batchPersistOrders(List<OrderEntity> orders, int batchSize) {
    if (orders == null || orders.isEmpty()) {
        return 0;
    }

    LOG.info(format: "Batch persisting {} orders with batch size {}", orders.size(), batchSize);
    int count = 0;

    for (int i = 0; i < orders.size(); i++) {
        em.persist(orders.get(i));
        count++;

        // Flush and clear every batchSize inserts
        if ((i + 1) % batchSize == 0) {
            em.flush();
            em.clear();
            LOG.debug(format: "Flushed and cleared at index {}", i + 1);
        }
    }
```

10.3 Leadership and Scenario Stabilization Contribution

As Group 1 lead, I coordinated integration across FIX and REST paths and enforced regression discipline, so changes were accepted only after stable end-to-end validation. In addition to module ownership, I contributed to scenario stabilization work, especially around stress behaviour and cancel-heavy flows. This included aligning lifecycle write timing with scenario expectations and driving repeated suite execution until behaviour remained consistent across runs. The outcome was not only a functioning module, but a dependable one under repeated stress validation.

11. Extended Discussion: Data Integrity and Lifecycle Guarantees

11.1 The Three Integrity Requirements

The order service is built on a lifecycle-first integrity model. Every accepted order must preserve three conditions from intake to terminal state.

1. Identity integrity: each order must remain uniquely identifiable throughout ingestion, persistence, and downstream updates.
2. Quantity integrity: filled quantity and remaining quantity must remain mathematically consistent with original order quantity.
3. State integrity: lifecycle transitions must be valid, and illegal transitions must be rejected deterministically.

These guarantees are essential in asynchronous trading workflows, where timing overlap can hide defects until later. If any one of these three breaks, the system can produce incorrect client views, poor auditability, and unstable test outcomes.

11.2 Canonical Persisted State as Source of Truth

A foundational design decision in this project is that persisted state is canonical. An operation is not considered complete until lifecycle state is durably written. This improves repeatability, debugging, and operational confidence because analysis is based on durable records, not transient runtime snapshots.

This decision also improves behaviour under stress. In high-concurrency windows, in-memory ordering can briefly diverge from eventual settled state. By treating persistence as authoritative, the system maintains a consistent reference point across retries, recovery, validation runs, and client queries.

11.3 Cancel and Replace as Controlled State Machines

Cancel and replace are implemented as guarded transitions, not generic updates. This matters because requests can arrive while matching is still progressing, and naive mutation logic can create invalid state windows. Guarded transitions enforce lifecycle rules at processing time, based on fresh state checks, and reject invalid operations on terminal orders.

This design gives predictable behaviour under race-prone conditions and keeps lifecycle semantics intact. Instead of treating order records as freely editable rows, the system treats them as stateful entities with explicit transition contracts.

12. Throughput, Scalability, and Reliability Engineering

12.1 Batch Operations and Scaling Considerations

Although this is an academic simulator, stress scenarios already emulate realistic burst behaviour. Batch insert and bulk update paths are therefore practical engineering requirements, not premature optimization. As order volume increases, persistence efficiency becomes a dominant factor, often before CPU becomes the bottleneck.
The current design is scale-aware for this phase because:

1. high-frequency lookup dimensions are indexed,
2. bulk operations reduce per-row overhead, and
3. lifecycle updates follow explicit, predictable write paths.

For next-stage scaling beyond current workloads, the recommended progression is:

1. query-trace-driven composite index tuning,
2. partitioning strategy for historical rows, and
3. formal retention and archival policy.

12.2 Scenario-Driven Engineering and Stabilisation Philosophy

A major quality strength in this project is scenario-first validation. Unit tests verify local logic, but true lifecycle correctness emerges only when protocol handling, orchestration, persistence, and stress timing are evaluated together. End-to-end scenarios exposed edge behaviours that isolated tests cannot reliably reproduce.
Stabilization followed three disciplined principles:

1. do not trust a single green run,
2. isolate flaky behaviour and tighten assertions around invariants, and
3. rerun full regression after each meaningful change.

This is a high-quality engineering practice and a strong indicator of delivery maturity.

13. Risk Register and Mitigation Plan

Risk	Likelihood	Impact	Mitigation
Race-sensitive cancel/replace behavior under concurrent load	Medium	High	Expand transition assertions around eventual consistency windows and revalidate under stress loops
Query slowdown as dataset grows	Low-Medium	Medium	Introduce profiling-based index review at release checkpoints
Regression from runner script or infra path changes	Low	High	Enforce mandatory full-suite gate after script/infrastructure modifications
Limited observability for slow persistence operations	Medium	Medium	Add latency percentile tracking and alerts for critical persistence paths

This register should be treated as a living artifact. As load patterns evolve, likelihood and mitigation priority should be updated based on measured behaviour, not assumptions.

14. Design Quality Evaluation

14.1 Strengths

The implementation demonstrates strong architectural clarity and practical correctness discipline. Key strengths are:

1. clear separation of concerns across intake, orchestration, matching, and persistence,
2. lifecycle state treated as a durable business contract,
3. index strategy aligned to observed operational query patterns,
4. batch-ready persistence paths for burst resilience, and
5. scenario-first validation that goes beyond local correctness checks.

14.2 Areas for Further Development

The current design is solid for project scope, but further maturity can be achieved through:

1. deeper transaction-level stress verification under higher concurrency profiles,
2. data-driven composite index tuning using measured traces, and
3. production-grade historical data lifecycle management (partitioning and archival jobs).

14.3 Acceptability of Current Limitations

These limitations are acceptable for the current stage because they are hardening opportunities, not structural weaknesses. The core architecture is extensible and can accommodate these improvements incrementally without redesigning the lifecycle model.

15. Conclusion

This project delivered a complete and stable order lifecycle implementation for a FIX-based exchange backend, covering intake, orchestration, matching, persistence, and operational controls through both FIX and REST paths. The key achievement was not only functional completeness, but lifecycle consistency: accepted orders are durably recorded, matching outcomes are reflected through deterministic status and quantity updates, and cancel/replace flows are guarded to prevent invalid terminal-state transitions. As a result, the system maintains a coherent and query able order state throughout processing rather than relying on transient in-memory behaviour.

From an individual contribution perspective, my role as G1-M4 was central to the reliability of the final system. I owned persistence and database integration, including lifecycle-aware entity modelling, schema and index design, DAO abstraction, and batch-capable write paths for stress conditions. In addition, as Group 1 leader, I coordinated integration boundaries and validation discipline across team modules, so the implementation remained consistent end to end. This combined ownership of both architecture and delivery process helped convert component-level work into a coherent, testable pipeline.

Finally, the system was validated through repeated external scenario execution, including stress-heavy flows where lifecycle drift and race sensitivity are most likely to appear. This gave confidence that the implementation is not only theoretically correct, but operationally stable under sustained end-to-end testing. The project therefore concludes with a strong technical baseline: durable lifecycle truth, guarded state transitions, and regression-backed stability suitable for academic demonstration and future extension